GovDocs AI

AI-Powered Government Document Intelligence Platform

Recommended Project Architecture — B.Tech CSE (AI & ML) Capstone

1. Architecture Overview

GovDocs AI is designed as a document-intelligence and Retrieval-Augmented Generation (RAG) platform. The system has two primary pipelines: a Document Processing Pipeline that runs when documents are ingested, and a Query & Answer Pipeline that runs whenever a user asks a question. A structured eligibility layer is added for scheme-specific rule matching.

```
USER
  |
  v
REACT FRONTEND
  | Chat • Scheme Dashboard • Eligibility • Document Viewer
  v
FASTAPI BACKEND
  |
  +----------------------+---------------------+
  |                      |                     |
  v                      v                     v
DOCUMENT PIPELINE    QUERY PIPELINE       USER / PROFILE
  |                      |
  v                      v
PDF TYPE DETECTION    QUERY PROCESSING
  |                      |
  +----------+           v
             |      QUERY EMBEDDING
     +-------+-------+       |
     |               |       v
  Native          Scanned  VECTOR SEARCH
     |               |       |
 pdfplumber      Tesseract  v
     |               |    TOP-K CHUNKS
     +-------+-------+       |
             |               v
             v          RAG PROMPT BUILDER
       TEXT + PAGE MAP       |
             |               v
       +-----+-----+    GEMINI / OLLAMA
       |     |     |         |
       v     v     v         v
    CLASS.  NER  SUMMARY  GROUNDED ANSWER
       |     |     |         |
       +-----+-----+         v
             |          SOURCE + PAGE
             v          + ELIGIBILITY
       EMBEDDINGS
             |
             v
       FAISS / CHROMADB
             |
             v
        POSTGRESQL
 Documents • Entities • Rules • Users • Queries
```

2. Core Design Principle

The project should not be treated as a simple PDF summarizer. The central design is: **Official Source → Document Intelligence → Structured Scheme Knowledge → Eligibility Engine → RAG → Personalized Answer.**

RAG reduces hallucination risk by grounding answers in retrieved document evidence. It should not be described as guaranteeing zero hallucinations.

3. Document Processing Pipeline

```
Government PDF
      |
```

```
      v
Detect PDF Type
      |
      +---- Native PDF ----> pdfplumber / PyMuPDF
      |
      +---- Scanned PDF ---> Tesseract OCR
                               |
                               v
                  Clean Text + Page Mapping
                               |
                               v
                      Text Chunking
                               |
              +----------------+----------------+
              |                |                |
              v                v                v
        Classification       NER          Summarization
              |                |                |
              +----------------+----------------+
                               |
                               v
                    Generate Embeddings
                               |
                    +----------+----------+
                    |                     |
                    v                     v
               FAISS / Vector DB     PostgreSQL
               chunks + metadata     structured data
```

4. Query & RAG Pipeline

```
User Question
     |
     v
Question Embedding
     |
     v
Semantic / Vector Search
     |
     v
Top-K Relevant Chunks
     |
     v
Document + Page Metadata
     |
     v
RAG Context Builder
     |
     v
Gemini API / Ollama
     |
     v
Grounded Answer
     |
     +--> Source Document
     +--> Page Number
     +--> Relevant Section
     +--> Eligibility Verdict
```

5. Structured Eligibility Engine

For schemes, eligibility should be represented as structured rules rather than relying entirely on an LLM. This makes the result more explainable and testable.

```
User Profile + Scheme Eligibility Rules
                  |
                  v
             Rule Engine
                  |
        +---------+---------+
        |         |         |
        v         v         v
       Age      Income     State
        |         |         |
        +---------+---------+
                  |
                  v
       PASS / FAIL per criterion
                  |
                  v
       Explainable Eligibility
```

```
            |
            v
Source Page Citation
```

6. Database Design

Table	Important fields
documents	id, title, document_type, source_url, file_path, publication_date, status
document_chunks	id, document_id, chunk_text, page_number, chunk_index
entities	id, document_id, entity_type, entity_text, page_number
schemes	id, document_id, scheme_name, ministry, objective, benefit, summary
eligibility_rules	id, scheme_id, attribute, operator, value, source_page
users	id, name, age, state, occupation, income, category
queries	id, user_id, question, answer, created_at

7. Recommended Technology Stack

Layer	Recommended technology
Frontend	React
Backend	Python + FastAPI
PDF extraction	PyMuPDF / pdfplumber
OCR	Tesseract OCR
Text processing	Python
Embeddings	Sentence Transformers — all-MiniLM-L6-v2
Vector database	FAISS (or ChromaDB)
Classification	scikit-learn
NER	spaCy
LLM	Gemini API or Ollama
Database	PostgreSQL

8. Three Levels of Intelligence

Level	Question answered	Main components
Understand	What is inside the document?	OCR, NER, classification, summarization
Retrieve	Where is the information I need?	Embeddings, semantic search, RAG, citations
Reason	What does this information mean for me?	Eligibility engine, profile matching, explainable rules

9. Recommended MVP

- Upload a government PDF.
- Extract text from native and scanned PDFs.
- Preserve document page numbers during extraction and chunking.
- Classify the document type.
- Extract named entities and generate a concise summary.
- Create embeddings and store chunks in a vector database.
- Allow natural-language questions over the document.

• Return answers with document and page citations.

• Provide a basic eligibility checker for scheme documents.

• Evaluate retrieval, answer, and classification accuracy on a held-out test set.

10. Demo Flow for Final Presentation

```
1. Upload a 30–50 page government scheme PDF.
2. System detects whether the PDF requires OCR.
3. Text is extracted while preserving page numbers.
4. AI classifies the document and extracts key entities.
5. A short scheme summary is generated.
6. User asks: “Can a final-year engineering student apply?”
7. Semantic search retrieves the relevant eligibility clauses.
8. RAG generates a grounded response.
9. UI displays the answer and exact source page.
10. User enters their profile and receives an explainable eligibility result.
```

11. Evaluation

Use approximately 30–40 real questions with known answers from the ingested documents. Measure retrieval accuracy (correct source/page retrieved), answer accuracy, and document classification precision/recall. For the eligibility engine, additionally measure rule-level correctness against manually verified eligibility cases.

12. Reliability Considerations

• Always retain the official source URL and document date/version.

• Preserve page numbers from OCR and native PDF extraction through every downstream stage.

• Show citations for important claims.

• Keep old document versions instead of overwriting them.

• Use an admin/reviewer step before publishing automatically extracted scheme rules in a production version.

• Treat AI-generated answers as grounded assistance, not as a replacement for the official government document.